

Timer Lawn: Solving the Timer Wheel Overflow Problem for Large Scale and High Throughput Systems

Adam Lev-Libfeld

NextSilicon Ltd.

Tel-Aviv, Israel

adam.lev.libfeld@nextsilicon.com

Abstract—Timer facilities sit at the core of operating systems [1], network protocols, and real-time and stream-processing systems. The standard structure, the timing wheel, is asymptotically optimal but is bounded by a maximum TTL fixed in advance: exceeding it forces an $O(n)$ rebuild, the overflow problem. This paper presents Lawn, a field-tested timer structure that keys its queues by time-to-live (TTL) rather than by expiration time, so its span is unbounded and it never rebuilds. We give the algorithm, prove it correct, and evaluate an allocation-free implementation (Lawn2) against a production tickless hierarchical timing wheel. Lawn2 holds memory flat across any TTL span, keeps the idle per-tick check at $O(1)$ (about 1 ns against the wheel’s cascade cost, which climbs to $\sim 747 \mu\text{s}$ at 120 million timers), deletes about 3.8 times faster at scale, and drains due timers 3.8 to 20.7 times faster the more TTLs cluster. These gains are conditional: once distinct TTLs exceed roughly 2 to 6% of live timers the per-tick cost becomes $O(t)$ and Lawn loses to the wheel. We state that boundary quantitatively and report where Lawn should and should not be used.

Keywords—Stream Processing, Timing Wheel, Dehydrator, Callout facilities, protocol implementations, Timers, Timer Facilities, Lawn.

I. Introduction

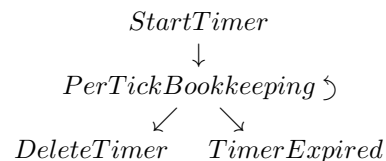
The current state-of-the-art timer structure, the timing wheel, is asymptotically optimal, but under some use cases it underperforms: when the maximum TTL is unpredictable, or the needed tick resolution may change, a wheel must either be sized past its real need or rebuilt at $O(n)$ cost when it overflows. Lawn addresses this by keying queues on TTL rather than on absolute expiration time, giving an unbounded span with no rebuild, under a single requirement: timer TTLs are drawn from a bounded set of discrete values.

The TTL-keyed queue-map idea, and the name Lawn, were introduced in an earlier preprint by the same author [2]. What is new here is the evaluation: an allocation-free implementation (Lawn2), a differential correctness gate across four independently written implementations, and a directly measured t/N crossover map (Section IV) showing exactly where the advantage holds and where it does not, alongside memory behavior at scale, workload

characterization, and a sweep to hundreds of millions of timers.

A. Model

Following prior work [3], [4], [5], a timer store is described by four routines, one per stage in a timer’s life cycle:



StartTimer(TTL, id, payload) registers a timer that expires after TTL, keyed by a client-supplied id, optionally carrying a payload (a callback or data returned on expiry). PerTickBookkeeping() runs every tick, expiring any due timer and invoking TimerExpired on it. DeleteTimer(id) cancels a live timer before it expires. TimerExpired(id) removes all trace of a timer and runs its payload. Payload cost varies by deployment and is $O(1)$ -storable in a hash map with parallel callback handling, so this paper disregards it.

II. Background and Related Work

The Timing Wheel [6] provides $O(1)$ insertion, deletion, and expiration in the common case, which is asymptotically optimal. Its limit is the maximum TTL and resolution combination fixed at construction: a timer past that range either forces a resize or must be tracked across wheel cycles. The Hierarchical Timing Wheel [3] widens the range with multiple levels at the cost of periodic cascade work and memory overhead. There are two correct ways to handle an overflowing timer, both $O(n)$ in the worst case. The first resizes the wheel to cover the larger range (Algorithm 1), an $O(n)$ cost on any overflowing insertion. The second (multi-pass) stores a per-timer cycle count and decrements it each pass, moving the $O(n)$ penalty onto PerTickBookkeeping. For large timer counts the common workaround is to over-provision resolution and range so as to postpone the rebuild.

Algorithm 1 Timer Wheel Resize (overflow)

```

1: function StartTimer( $id, ttl, payload$ )
2:   if  $ttl > len(Wheel)$  then
3:     MissingSlots  $\leftarrow ttl - len(Wheel)$ 
4:     ExtWheel  $\leftarrow$  array of length  $len(Wheel) +$ 
       MissingSlots
5:     for Slot, SlotIndex in Wheel do
6:       NewSlotIndex  $\leftarrow (len(Wheel) + SlotIndex)$ 
       mod  $len(ExtWheel)$ 
7:       ExtWheel[NewSlotIndex]  $\leftarrow$  Slot
8:     Wheel  $\leftarrow$  ExtWheel
9:     slot  $\leftarrow (current\ time + ttl) \bmod len(Wheel)$ 
10:    append  $payload$  to Wheel[slot]
11:    TimerHash[id]  $\leftarrow (payload, slot)$ 

```

A separate line of work, Calendar Queues [4], hashes on absolute expiration time into a resizable bucket array rather than on TTL duration, sharing the model’s shape but not its overflow-avoidance mechanism. The general principle of bucketing by a discretized TTL to bound storage has also appeared outside the timer literature: Time-Bucketed Balance Records [7] apply the same coalesce-within-a-bucket idea to bound per-account storage for TTL-bearing tokens, a different domain with the same underlying mechanism. Concurrent and NUMA-aware timer and priority-queue implementations [8], [9], [10], [11] address a contention and locality problem orthogonal to overflow; Lawn as evaluated here is not lock-free, and concurrency is left as future work.

III. The Lawn Data Structure

A. Intended Use and Core Assumption

Lawn was first built for a large-scale stream-processing geographic-intersection product on a FastData [12] model, where inputs use a very limited range of TTLs relative to the number of concurrent timers. This is the load-bearing assumption of the whole approach:

Unique TTL Count $\ll$ Concurrent Timer Count.

When it holds, TTL bucketing yields the results in Section IV. When it fails, Lawn degrades to $O(n)$ per tick and loses to the wheel. The assumption is a design argument, not a measured field claim: when TTLs come from an algorithmic or configuration-driven source (a fixed backoff schedule, a few cache-tier classes), the set of possible values is bounded by construction and does not grow with the number of live timers. We did not instrument the deployments in Section VI-A to confirm their distinct-TTL fraction, and flag that as open verification. Section IV measures where the crossover lies.

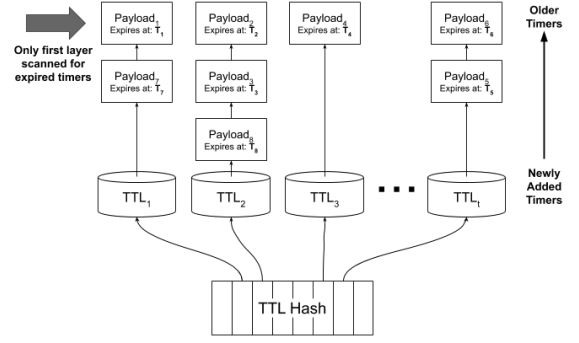


Fig. 1. A schematic view of the data structure components.

B. The Structure

Lawn is a hash of sorted sets¹, much like the Timing Wheel, but keyed on the timer TTL rather than on expiration time. Timers share a set based only on their TTL, regardless of arrival time, and within each set are naturally ordered by arrival (Figure 1). Because two timers in one bucket share a TTL, any timer already queued expires before a newly added one, so the queue stays sorted by expiration for free. The structure is analogous to blades of grass (hence the name): each blade grows from the roots up, and every *Tick* the overgrown tops (expired timers) are mowed to the current-time level.

C. Algorithm

Note the *closest expiration* guard in PerTickBookkeeping: a tick on which nothing is due returns in $O(1)$ without scanning the buckets. This empty-tick fast path matters in practice because most ticks expire nothing.

D. Two Implementations: Reference Lawn and Lawn2

The same algorithm admits two C implementations that produce identical expiry schedules (verified by the differential test of Section IV) and differ only in node ownership and cancellation addressing. Reference Lawn favors readability: the store owns each timer node, copies the client’s key, and indexes timers in a hash map, so a timer is cancellable by an arbitrary opaque key, at the cost of one allocation, one key copy, and two hash-map operations per insertion. Lawn2 removes that per-operation allocation: nodes are intrusive and caller-owned (embedded in the client object), so insertion neither allocates nor copies a key, deletion is $O(1)$ through the node’s own links (dropping the id hash map), and the TTL-to-queue map uses open addressing with no per-entry allocation. All performance claims in this paper refer to Lawn2.

¹The TTL ‘buckets’.

Algorithm 2 The Lawn Data Store

Precondition:

```

1: id - a unique identifier of a timer.
2: tttl - a whole product of TickResolution representing
   the amount of time to wait before triggering the given
   timer payload action.
3: payload - the action to perform upon timer expiration.
4: current time - the local time of the system as a whole
   product of TickResolution

5: function InitLawn()
6:   TTLHash  $\leftarrow$  new empty hash set
7:   TimerHash  $\leftarrow$  new empty hash set
8:   closest expiration  $\leftarrow \infty$ 

9: function StartTimer(id, tttl, payload)
10:  endtime  $\leftarrow$  current time + tttl
11:  T  $\leftarrow$  (endtime, tttl, id, payload)
12:  TimerHash[id]  $\leftarrow T$ 
13:  if tttl  $\notin$  TTLHash then
14:    TTLHash[tttl]  $\leftarrow$  new empty queue
15:  TTLHash[tttl].insert(T)
16:  if endtime < closest expiration then
17:    closest expiration  $\leftarrow$  endtime

18: function PerTickBookkeeping()
19:  if current time < closest expiration then
20:    return
21:  closest expiration  $\leftarrow \infty$ 
22:  for queue  $\in$  TTLHash do
23:    T  $\leftarrow$  peek(queue)
24:    while Tendtime < current time do
25:      TimerExpired(Tid)
26:      T  $\leftarrow$  peek(queue)
27:    if Tendtime < closest expiration then
28:      closest expiration  $\leftarrow T_{endtime}$ 

29: function TimerExpired(id)
30:  T  $\leftarrow$  TimerHash[id]
31:  DeleteTimer(T)
32:  do Tpayload

33: function DeleteTimer(id)
34:  T  $\leftarrow$  TimerHash[id]
35:  if Tendtime = closest expiration then
36:    closest expiration  $\leftarrow \infty$ 
37:  TTLHash[Tttl].remove(T)
38:  TimerHash.remove(T)
39:  if TTLHash[Tttl] is empty then
40:    TTLHash.remove[tttl]

```

E. Correctness

Let T be a timer with TTL t_{ttl} started at t_{start} ; we show it is expired at $t_{start} + t_{ttl}$. Let Q_{ttl} be its queue, $t_{current}$ the current time, and $t_{expiry} = t_{start} + t_{ttl}$. Base case: at t_{start} , T is inserted into Q_{ttl} with expiry t_{expiry} . Inductive step: assume $T \in Q_{ttl}$ at $t_{current} < t_{expiry}$. If $t_{current} + 1 < t_{expiry}$, T remains, since PerTickBookkeeping only expires timers with expiry $\leq t_{current} + 1$. If $t_{current} + 1 \geq t_{expiry}$, T is expired. By induction T is expired at t_{expiry} . Since each timer is stored exactly once in its bucket and buckets are independent, per-bucket correctness gives whole-structure correctness.

F. Complexity

Lawn is dense by design: every timer is stored exactly once, a new trigger adds at most one bucket, and empty buckets are removed, so space is $O(n)$ (confirmed empirically in Section IV). StartTimer and DeleteTimer are amortized $O(1)$, with an $O(n)$ worst case on a hash resize as is standard for hash-based structures. An empty tick is $O(1)$ via the *closest expiration* guard. A tick that expires timers scans the head of each of the t live buckets, so its cost is $O(t+k)$ for k expired timers, worst case $O(n)$ when all timers fall due at once. When t is bounded ($t = O(1)$, the assumption above) the per-tick cost is $O(1)$; otherwise it is $O(t)$ and Lawn is worse than a wheel. This dependence on t is the algorithm's defining trade-off and is measured directly in Section IV. Table I places Lawn against both wheel overflow schemes.

Operation	TW (Resize)	TW (Multi-Pass)	Lawn
StartTimer	$O(n)^*$	$O(1)$	$O(1)^\dagger$
PerTick	$O(1)$	$O(n)^\ddagger$	$O(t)^\S$
DeleteTimer	$O(1)$	$O(1)$	$O(1)^\dagger$
TimerExpired	$O(1)$	$O(1)$	$O(1)$
space	$O(n)$	$O(n)$	$O(n)$

* on overflow, otherwise amortized $O(1)$. $\dagger$ amortized, worst case $O(n)$ on a resize. $\ddagger$ worst case, from the multi-pass cascade. $\S$ $O(1)$ empty tick and mean when $t = O(1)$, $O(n)$ worst case.

TABLE I

Runtime complexity comparison. The Multi-Pass $O(n)$ penalty falls on PerTick (the cascade), Lawn's $O(t)$ on the bucket scan.

Space is $O(n)$, bounded above by the Hashed Hierarchical Timing Wheel, whose multi-level structure can chain several wheels to a single timer. Because TTL buckets are independent, a Lawn can be partitioned across workers by bucket with no cross-bucket synchronization; we evaluate a sharded, coarse-locked version in Section V and find no concurrency win under coarse locking. We make no measured cache-locality claim: the memory results below are footprint only.

IV. Experimental Evaluation

A. Methodology

We benchmark four C implementations behind one interface, all in the project repository [13]: the reference Lawn (src/lawn.c), the allocation-free Lawn2

(src/lawn2.c), a production tickless Hierarchical Timing Wheel (Timer Wheel, William Ahern’s timeout.c [14]), and a Naive single-level growing ring. Every wheel result in this paper refers specifically to that one cascading, tickless implementation, not to timing wheels as a class; we also differentially verified two further baselines, a non-cascading Linux-style wheel and a binary min-heap, whose numbers occupy a materially different point in the design space and are reported in the repository. A differential correctness gate runs first: all implementations must produce byte-identical per-tick expiry schedules across fixed, discrete, and continuous TTL distributions, with and without deletions and for large TTLs that force cascades, before any measurement is accepted.

The evaluation reaches 2×10^8 concurrent timers across three TTL regimes: fixed ($t = 1$), discrete ($t \ll n$), and continuous uniform ($t \approx n$). Each measurement point runs in its own forked child process, so no two share allocator or cache state. Timing uses the finest monotonic clock available (CLOCK_UPTIME_RAW, ~ 41 ns quantized), with sub-40 ns operations batched in 256 to lift them off the clock floor; fixed seeds (base 1234), 7 runs and 2 warmups (3/1 at 10^6 and above). The TTL-span baseline of 1024 ticks was chosen by a dedicated empty-tick sweep from 128 to 65 536 ticks, not picked arbitrarily: the wheel’s cost is genuinely non-monotonic in span (a reproducible dip near 2048 and spike near 4096, a property of its level structure), and 1024 is the smallest span past the steep small-span regime that sits on neither. Hardware: Apple M4 Pro (12 cores, 8 performance + 4 efficiency), 24 GB RAM, macOS 26.3, Apple clang 21.

B. Overflow-Free Memory

Lawn’s defining property is that its footprint is independent of TTL span. Figure 2 sweeps total memory against TTL span from 128 to 524,288 ticks (a 4,096-fold range) at one hundred thousand timers. Lawn, Lawn2, and the tickless wheel are flat; the naive single-level ring grows from 4.2 MB to 29.3 MB (7.0 times) as its ring must expand to cover the widening span, an $O(n)$ rehash of the same kind as the wheel-resize overflow algorithm (Algorithm 1). This is the overflow problem made visible: Lawn never pays it, because it buckets on TTL value rather than absolute time. At a fixed, modest span Lawn2 is also lighter per timer: about 48 bytes against the wheel’s about 88 (1.8 times) at scale, growing more slowly (Figure 3). At one thousand timers Lawn2’s fixed overhead makes it heavier, so the per-timer advantage is a scale effect.

C. Constant-Time Idle Tick

Most ticks expire nothing, so the cost of an idle tick is a tax paid on every clock tick regardless of load. Lawn’s *closest expiration* guard makes it $O(1)$. Figure 4 extends this to 200 million timers: Lawn2 holds at about 1 ns at every scale, while the cascading wheel’s empty-tick cost

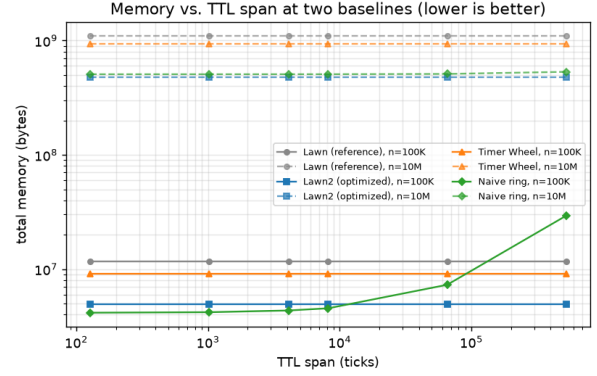


Fig. 2. Total memory vs. TTL span at one hundred thousand timers (C, bytes, lower is better). The naive ring overflows while Lawn, Lawn2, and the wheel stay flat.

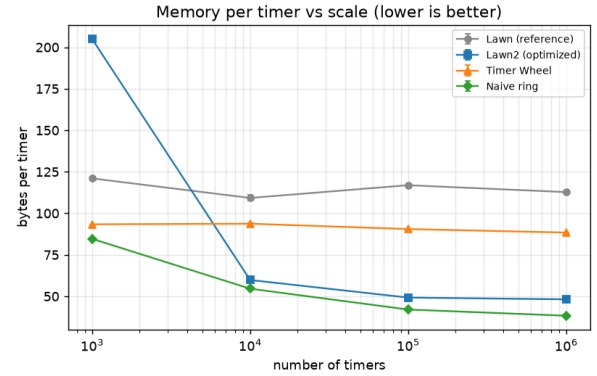


Fig. 3. Memory per timer across scales at a fixed 1024-tick span (C, bytes, lower is better).

grows with absolute timer count, $6.6 \mu\text{s}$ at one million, $46.1 \mu\text{s}$ at ten million, $621.1 \mu\text{s}$ at one hundred million, and $747.2 \mu\text{s}$ at 120 million, more than five orders of magnitude higher. Per-timer memory stays flat for every implementation across the same range (Lawn2 converges to 48.0 B and holds with zero variance through 200 million), so this cost is specific to the wheel’s cascade, not to scale in general. This is the empirical form of the overflow problem: the wheel does work proportional to elapsed time and level structure, Lawn does not.

D. Insertion and Deletion

Figure 5 shows deletion latency: Lawn2 is 3.2 to 3.8 times faster than the wheel (23.4 ns vs. 88.5 ns at one million, 4.5 ns vs. 14.3 ns at one hundred thousand), the ratio roughly constant so the absolute gap widens with scale. On insertion Lawn2 is 1.4 to 1.6 times faster at scale (4.9 ns vs. 8.0 ns at one million); the unsorted naive ring leads at ~ 2.3 ns and the reference Lawn trails at ~ 106 ns from its per-insert allocation and key copy, which is why the performance claims use Lawn2.

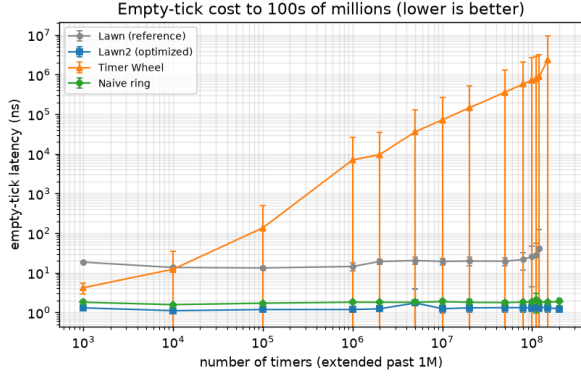


Fig. 4. Empty-tick latency vs. timer count, extended to hundreds of millions (C, ns). Lawn2 stays flat while the wheel’s cost keeps climbing.

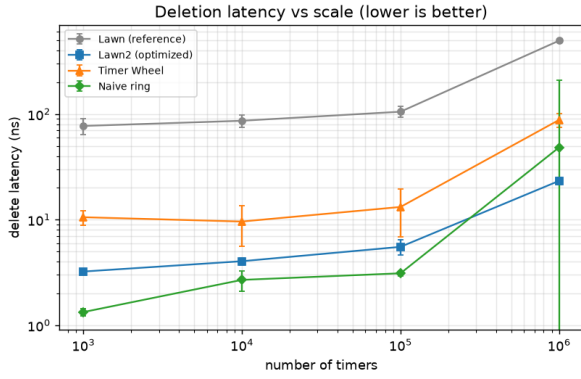


Fig. 5. Deletion latency across scales (C, ns).

E. Draining Due Timers

A tick that drains due timers does the real work of firing callbacks. Both Lawn2 and the wheel must inspect every live bucket or level on such a tick, and Lawn2 does so more cheaply. Figure 6 measures per-tick expiry at one hundred thousand timers under four TTL distributions (Lawn2 vs. the wheel): a single shared TTL, 556 ns vs. 11,479 ns ($\sim 20.7\times$); ten distinct TTLs, 3,407 ns vs. 13,325 ns ($\sim 3.9\times$); bursty arrivals, 1,530 ns vs. 10,292 ns ($\sim 6.7\times$); uniform, 3,725 ns vs. 14,056 ns ($\sim 3.8\times$). Lawn2 wins on every distribution, most when TTLs cluster. Even at one distinct TTL per timer ($t/N = 1$) Lawn2 still drains faster than the wheel, so it is the idle-tick amortization, not the draining work, that flips at the crossover below.

F. Realistic Mixed Workload

Isolated single-operation sweeps can hide interactions a production facility exposes, where insertions, cancellations, and tick draining interleave continuously. We ran a mixed workload: pre-fill n background timers, then execute 5,000 iterations, each draining a tick and, independently, inserting a new timer with 50% probability and cancelling a live one with 1% probability, timing every individual

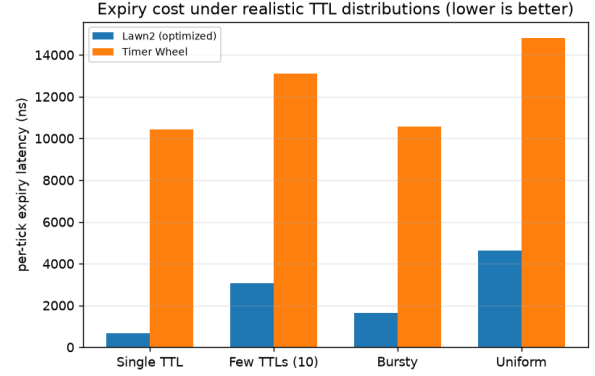


Fig. 6. Per-tick expiry latency, Lawn2 vs. the wheel, under four realistic TTL distributions (C, ns, lower is better).

operation. Figure 7 plots mean and 99th-percentile latency from one thousand to one million background timers. At one hundred thousand, Lawn2 beats the wheel on mean (97.4 ns vs. 205.5 ns) and the two are close on p99 (292 ns vs. 208 ns): the wheel’s mean is pulled up by rare, severe stalls rather than a higher typical cost, visible in its far larger maximum (186,583 ns vs. 6,083 ns). At one million the split sharpens, Lawn2 stays flat (102.3 ns mean, 375 ns p99) while the wheel’s mean grows past 2,152 ns with a 2.1 ms maximum, the same cascade cost now under a realistic load. Extended to the largest count each implementation reaches on the test machine (120 million for the wheel, 150 million for Lawn2), Lawn2’s mean holds between 68 and 84 ns while the wheel’s grows almost proportionally with N , confirming the pattern under a mixed workload rather than an isolated empty-tick sweep.

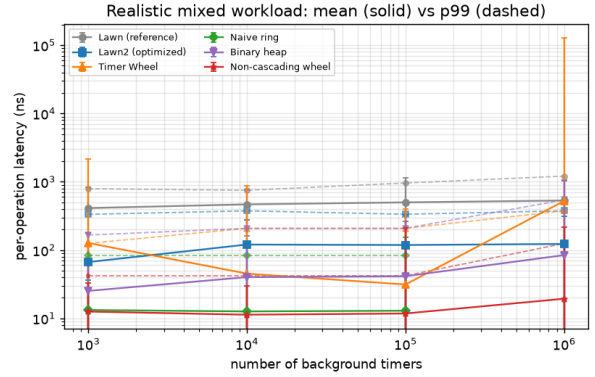


Fig. 7. Per-operation latency under a realistic mixed insert/cancel/tick workload, mean (solid) and p99 (dashed) (C, ns, lower is better).

G. The Distinct-TTL Crossover

The paper’s one hard boundary is measured directly in Figure 8, which plots Lawn2’s full-lifecycle speedup over the wheel (Wheel cost divided by Lawn2 cost) as the

distinct-TTL fraction t/N varies. Lawn2 wins while t/N is small and crosses over to losing at $t/N \approx 6.4\%$ ($N = 10^4$, about 643 distinct TTLs), 2.2% ($N = 10^5$, about 2234), 1.29% ($N = 10^6$, about 12920), and 0.71% ($N = 10^7$, about 71407). When nearly every timer has a distinct TTL ($t/N = 1$) Lawn2 is up to about 116 times slower. The crossover is about idle-tick cost amortized over the full lifecycle, not about draining work, which Lawn2 wins even at $t/N = 1$ (Section IV-E). Lawn2’s per-tick cost is $O(t + k)$ for t active buckets and k expiring timers; a hierarchical wheel’s is $O(L + k)$ for a small level count L independent of N . As t grows past a few percent of n , the balance tips. The boundary tightens as N grows because the wheel’s own per-tick cost rises with absolute timer count (the same effect seen in Figure 4), so a smaller distinct-TTL fraction suffices to overtake Lawn2.

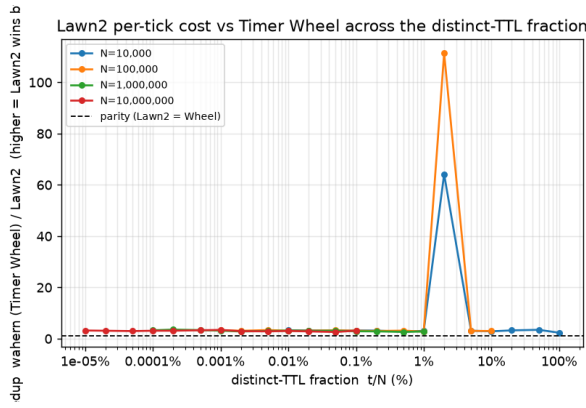


Fig. 8. Lifecycle speedup of Lawn2 over the Timer Wheel vs. the distinct-TTL fraction t/N (C). Higher means Lawn2 wins by more; below 1 the wheel wins.

V. Limitations

Lawn’s advantages are conditional, so its limits define where to use it.

TTL distribution. Performance depends directly on the distinct-TTL count t . When TTLs are continuous or highly variable, t grows toward n and PerTickBookkeeping approaches $O(n)$. Lawn2 loses to the wheel once t/N exceeds roughly 0.71 to 6.4% (tightening as N grows from 10^4 to 10^7) and is more than two orders of magnitude slower when nearly every timer has a distinct TTL. Lawn is a win precisely when distinct TTLs stay well under a few percent of live timers, the range a configuration-driven or otherwise bounded TTL set stays within.

Small-scale memory. At one thousand timers Lawn2’s fixed per-structure overhead makes it heavier per timer than the wheel; the memory advantage appears only at scale.

Concurrency. Under an external per-shard mutex applied identically to all implementations, single-threaded Lawn2 reaches 121 M ops/s against the wheel’s 90 M (the naive ring leads at 134 M). Adding threads under coarse

locking lowers aggregate throughput for every implementation on this machine (turbo and core-asymmetry effects), and Lawn2 keeps only a ~ 1.3 to $1.4\times$ edge at twelve threads (18.8 M vs. 14.1 M ops/s). We found no evidence of a large concurrency advantage for bucketed timers; realizing one would require a lock-free implementation, left to future work.

Measurement ceiling. The mixed-workload test is a single-machine measurement. On the 24 GB test machine it could not initially pass one million background timers without the process’s own footprint inducing swap, confirmed by kernel swap counters; after freeing other memory we re-ran with zero swap events and extended to one hundred million and beyond. This is not a guarantee that a differently loaded machine would show the same headroom at the same scale.

VI. Conclusion

Lawn is a simple, overflow-free timer structure that keys queues by TTL rather than expiration time, giving an unbounded span with no rebuilds. Its allocation-free implementation, Lawn2, holds memory flat across any TTL span and keeps the idle per-tick check at $O(1)$ against the wheel’s cascade cost, which climbs past $747\mu s$ at 120 million timers. It also deletes about 3.8 times faster at scale, inserts 1.4 to 1.6 times faster, uses about 1.8 times less memory per timer, and drains due timers 3.8 to 20.7 times faster under every TTL distribution tested. These gains hold only while distinct TTLs stay under roughly 2 to 6% of the live-timer count; past that, Lawn degrades to $O(n)$ per tick and loses to the wheel. Lawn is the right choice for large-scale systems whose timers are drawn from a bounded set of discrete TTLs, and the wrong one for continuous or near-unique TTL distributions.

A. Contributions

- **Data structure:** the TTL-keyed Queue-Map (Lawn) that removes the overflow problem, plus an allocation-free implementation (Lawn2).
- **Analysis:** correctness, and complexity with an explicit, measured account of the $O(t)$ per-tick dependence and the empty-tick $O(1)$ fast path.
- **Honest evaluation:** a differential-gated C benchmark of four implementations across scale, TTL span, distinct-TTL count, and workload, quantifying both where Lawn wins and the t/N crossover where it loses.
- **Deployment:** Lawn is in production use by several organizations, some independent of the author, including Redis Internals for streams and other internal timers [15], the ReDe Redis dehydration module [16], and Mellanox RDMA timers for Unified Communication X [17], with source available for the ReDe and Unified Communication X implementations.

B. Future Work

- Lock-free / per-bucket-lock Lawn: a genuinely concurrent implementation, and an evaluation on NUMA hardware (not attempted here).
- Formal verification of correctness using theorem provers or model checkers.
- Deterministic bulk-service queueing model: each bucket is drained in one batch per tick, so the honest target is a bulk-service queue rather than a single-item M/D/1 model; we leave its derivation and empirical validation open.
- Distributed implementations for geographically distributed systems.

References

- [1] A. M. Costello and G. Varghese, “Redesigning the bsd callout and timer facilities,” tech. rep., Washington University in St Louis, 1995.
- [2] A. Lev-Libfeld, “Lawn: an unbound low latency timer data structure for large scale, high throughput systems,” arXiv:1906.10860, 2019.
- [3] G. Varghese and A. Lauck, “Hashed and hierarchical timing wheels: efficient data structures for implementing a timer facility,” *IEEE/ACM transactions on networking*, vol. 5, pp. 824–834, December 1997.
- [4] D. Sleator and R. Brown, “Calendar queues: A fast $o(1)$ priority queue implementation for the simulation event set problem,” *Communications of the ACM*, vol. 31, Oct. 1988.
- [5] D. W. Jones, “An empirical comparison of priority-queue and event-set implementations,” *Commun. ACM*, vol. 29, pp. 300–311, Apr. 1986.
- [6] G. Varghese and T. Lauck, “Hashed and hierarchical timing wheels: Data structures for the efficient implementation of a timer facility,” *SIGOPS Oper. Syst. Rev.*, vol. 21, pp. 25–38, Nov. 1987.
- [7] S. Scovil and B. C. Nanjundappa, “Time-bucketed balance records: Bounded-storage ephemeral tokens for resource-constrained systems,” arXiv:2512.20962, 2025.
- [8] H. Hu, “Large-scale timer management,” US Patent no. US8307030B1.
- [9] Z. Zhou, I. D. Pashov, J. A. Silvera, and M. R. Cox, “On-demand scalable timer wheel,” US Patent no. US20140298073A1.
- [10] C. Giannoula, F. Strati, D. Siakavaras, G. Goumas, and N. Koziris, “Smartpq: An adaptive concurrent priority queue for numa architectures,” 2024.
- [11] A. Drebes, A. Pop, K. Heydemann, N. Drach, and A. Cohen, “Numa-aware scheduling and memory allocation for data-flow task-parallel applications,” *SIGPLAN Not.*, vol. 51, Feb. 2016.
- [12] A. Lev-Libfeld and A. Margolin, “Fast data - moving beyond from big data’s map-reduce,” in *GeoPython*, pp. 3–6, FHNW, June 2016.
- [13] A. Lev-Libfeld, “Lawn github repository,” <https://github.com/picotera/lawn>.
- [14] W. Ahern, “timeout.c: Tickless hierarchical timing wheel implementation,” <http://25thandclement.com/~william/projects/timeout.c.html>, 2014.
- [15] S. Sanfilippo, “Redis,” <https://redis.io/>, 2009.
- [16] A. Lev-Libfeld, “A redis element dehydration module,” <https://github.com/TamarLabs/ReDe>, 2017.
- [17] “Unified communication x,” <https://github.com/openucx/ucx/tree/master/src/ucs/time>, 2018.